SALUD VACUNAS · ARQUITECTURA

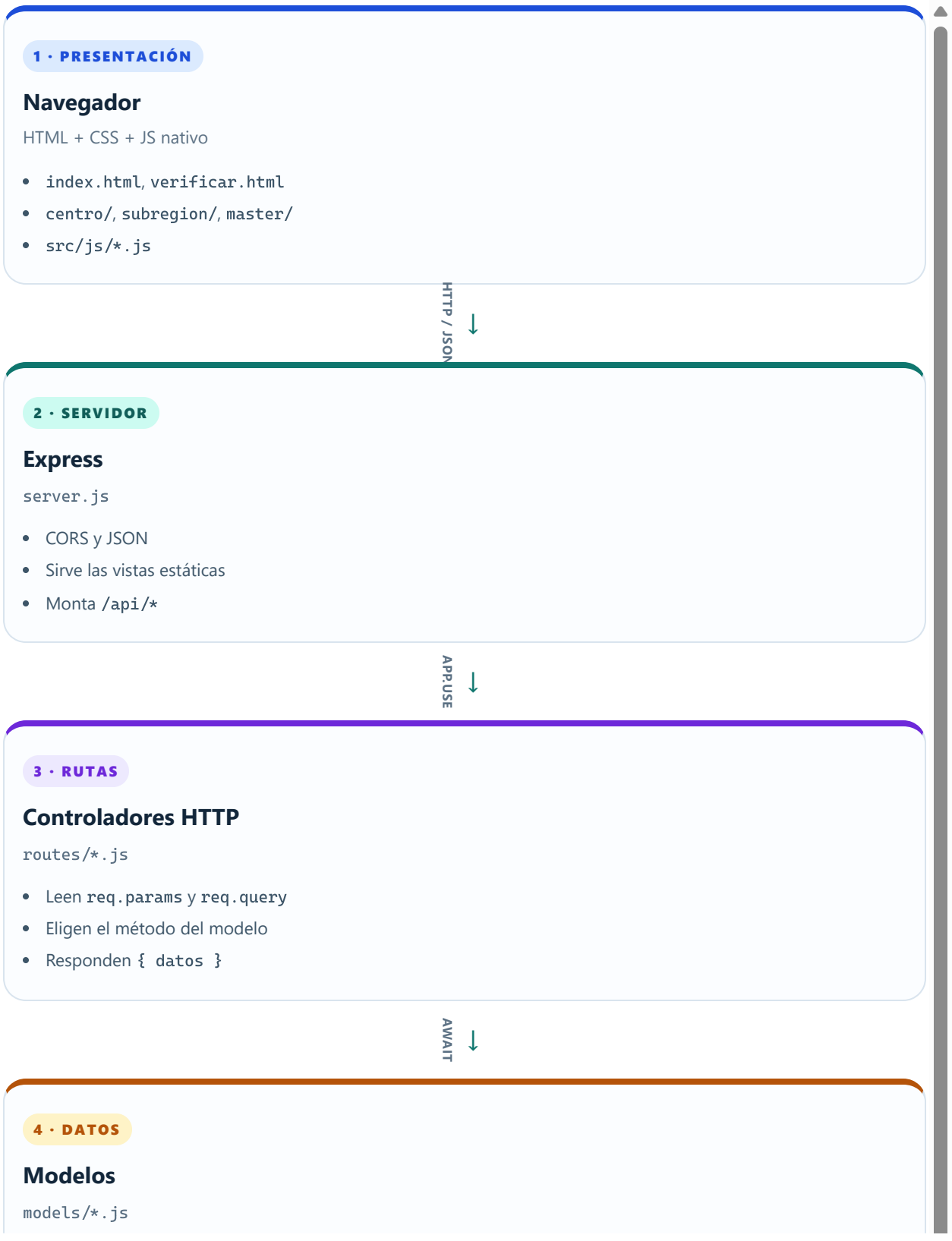
Cómo funciona el sistema, de la pantalla a MySQL

Recorrido visual de las capas, las seis tablas, las relaciones y los flujos principales: registrar una vacuna, emitir un certificado con exámenes y verificarlo públicamente.

5 capas | 6 tablas InnoDB | 7 recursos de API | SQL parametrizado | Transacciones | Sin frameworks de interfaz

1 Vista general por capas

Una petición entra por la interfaz, viaja por Express, la atiende una ruta, la ruta delega en un modelo y el modelo ejecuta SQL contra MySQL. La respuesta recorre el camino inverso hasta convertirse en JSON.



- Validan y mapean campos
- Construyen SQL seguro
- Promesas y transacciones

POOL.EXECUT ↓

5 · PERSISTENCIA

MySQL 8

base `salud_vacunas`

- 6 tablas InnoDB
- Claves foráneas y CHECK
- Índices por cédula, fecha y centro

El viaje de vuelta: MySQL devuelve filas → el modelo las convierte a objetos camelCase → la ruta las envuelve en JSON → la interfaz las pinta. Los modelos nunca escriben código HTTP y las rutas nunca escriben SQL.

Dos modos de uso del proyecto

ASPECTO	MODO DEMOSTRACIÓN	MODO SERVIDOR + API
Cómo se abre	`index.html` directamente (`file://`)	`npm start` y `http://localhost:3000`
De dónde salen los datos	Bloques `data-*` del HTML y `sessionStorage`	Base de datos MySQL
Lógica	`src/js/` (núcleo, datos, vistas)	`server.js` → `routes/` → `models/`
Estado actual	Funciona sin instalar nada	API lista; las vistas actuales todavía usan datos de demostración

Importante: la API existe y está conectada a MySQL, pero el JavaScript de las vistas todavía no hace `fetch()` a `/api`. Esa es la pieza que falta para pasar del boceto a la aplicación con datos reales.

Del HTTP al SQL en una línea

```
// POST /api/personas
routes/personas.js      router.post('/', ...)      // recibe req.body
  await personas.crear(req.body)                   // llama al modelo
```

```
    utiles.construirInsercion( ... )                    // mapea camelCase → columnas
    conexion.consulta('INSERT INTO personas ( ... ) VALUES ( ... )', valores)
      // mysql2 envía los valores como parámetros (?)
// respuesta: 201 { datos: persona }
```

2 Dentro de la capa `models/`

`models/index.js` es la puerta única: agrupa los seis modelos y la conexión. Cada archivo de modelo representa una tabla. Dos módulos de apoyo —`conexion.js` y `utiles.js`— evitan repetir pool, mapeo, filtros y validaciones.

models/index.js

Exporta todo lo que el resto del servidor puede usar

↓ REQUIRE / MODULE.EXPORTS

centros

`tabla centros`

Catálogo de centros, subregiones y dependencias.

personas

`tabla personas`

Usuarios, búsqueda por cédula y listado por centro.

vacunas

`tabla vacunas`

Catálogo de vacunas, obligatoriedad y lotes JSON.

dosis

`tabla dosis`

Aplicaciones, peso/altura, historial y estadísticas.

certificadosSalud

`tabla certificados_salud`

Emisión, verificación, vencimiento y transacción con exámenes.

examenes

`tabla examenes`

Exámenes de cada certificado; inserción individual o masiva.

↓ TODOS USAN

conexion.js

pool de mysql2, `consulta()`, `unaFila()`, `transaccion()`, `probarConexion()` y `cerrar()`.

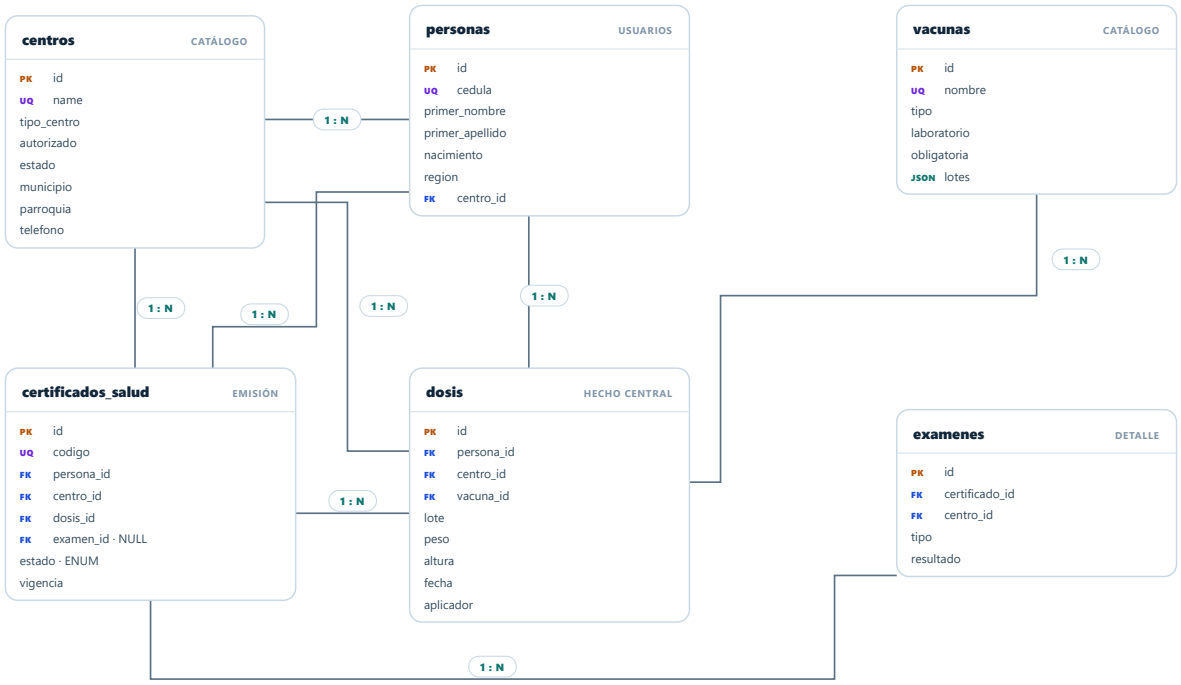
utiles.js

`filtros()`, `paginacion()`, mapeo camelCase, construcción de INSERT/UPDATE, validaciones y JSON.

Contrato común: todos los métodos devuelven promesas. Por eso las rutas pueden usar `async/await` sin saber si hay pool, transacción o reintento detrás.

3 Modelo entidad-relación

Seis tablas, un núcleo claro: **centros** y **personas** son el eje; **vacunas** alimenta **dosis**; y **certificados_salud** une persona, centro y dosis con uno o varios **exámenes**.



Leyenda: PK clave primaria · **FK** clave foránea · **UQ** único · **JSON** columna JSON. Las líneas indican relaciones 1 : N; las flechas no se dibujan para no saturar, pero la FK vive siempre en el lado "N".

Relación circular: `certificados_salud.examen_id` apunta al examen principal y `examenes.certificado_id` apunta de vuelta al certificado. MySQL no permite FK diferidas, así que el modelo inserta primero el certificado con `examen_id = NULL`, luego los exámenes y al final actualiza el examen principal. Todo dentro de una transacción.

No dibujado para no saturar: `dosis.centro_id`, `certificados_salud.centro_id` y `examenes.centro_id` también referencian a `centros`. Es decir, un centro aparece en todas las tablas operativas.

4 Flujo: registrar una vacunación

Es el flujo más frecuente del Perfil 1. El formulario envía una dosis aplicada con sus medidas, lote y aplicador; el modelo valida y hace un solo `INSERT`.

1

Formulario

El usuario elige persona, vacuna, lote y completa peso, altura y datos del aplicador.

`src/js/centro.js`

↓

2

Petición HTTP

Se envía el JSON al recurso de dosis.

`POST /api/dosis`

↓

3

La ruta delega

La ruta valida el cuerpo y llama al modelo; no escribe SQL.

`routes/dosis.js`

↓

4

Validar + insertar

`exigir()` comprueba campos; el INSERT va parametrizado. MySQL aplica FK y CHECK.

`models/dosis.js`

↓

5

Respuesta

`201 { datos: dosis }`. Si algo falla: 400 por validación o 500 por base de datos.

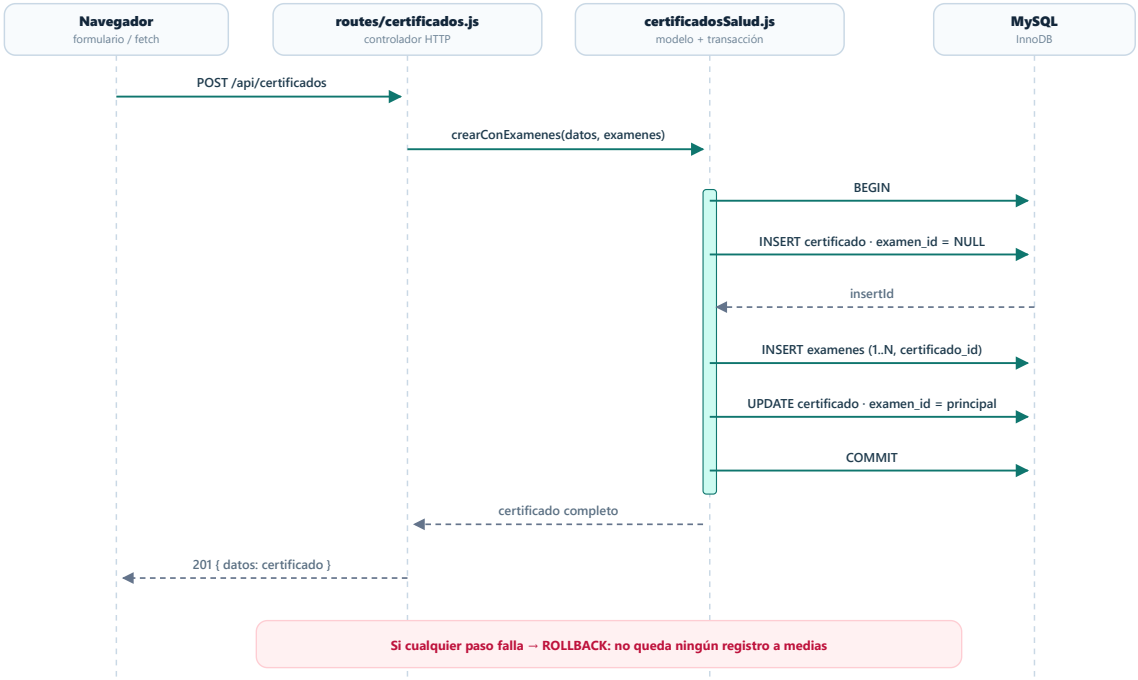
`JSON`

Regla del modelo: peso y `altura` se guardan en cada fila de `dosis`, no en `personas`. Así queda el historial antropométrico en el momento de cada aplicación.

Restricciones que protegen el registro: la persona, el centro y la vacuna deben existir (FK); `cantidad`, `peso` y `altura` deben ser positivos y estar en rango; y `lote_vence` no puede ser anterior a `fecha`.

5 Flujo: emitir un certificado con exámenes

Es la operación más delicada: toca dos tablas con una relación circular. `certificadosSalud.crearConExamenes()` la resuelve con una transacción: o se guarda todo, o no se guarda nada.



Por qué así: MySQL no admite claves foráneas diferidas. Insertar el certificado con `examen_id = NULL` rompe el círculo; luego se insertan los exámenes, que ya pueden apuntar al certificado; y al final se asigna el primero como examen principal.

Si algo falla: `conexion.transaccion()` ejecuta `rollback()` en el `catch` y libera la conexión en `finally`. El error se propaga a la ruta y esta responde con el estado HTTP adecuado. Nunca queda un certificado sin exámenes.

Después del commit: el modelo devuelve el certificado completo con sus exámenes mediante `obtenerCompleto()`, para que la interfaz no tenga que hacer una segunda consulta.

6 Flujo: verificar un certificado

Es el flujo público de `verificar.html`: cualquiera puede consultar un certificado por su código y ver si está vigente. No requiere sesión.

1

Consulta pública

El usuario introduce el código del certificado (o busca por cédula en el Perfil 2).

`verificar.html`

↓

2

Ruta de verificación

La ruta pública no exige autenticación y valida el código recibido.

`GET /api/certificados/verificar/:codigo`

↓

3

Buscar el certificado

`verificarPorCodigo()` consulta por el código único.

`models/certificadosSalud.js`

↓

4

Reunir el contexto

El resultado incluye a la persona, al centro, el id de la dosis, los exámenes y un booleano `vigente`.

`SELECT + JOIN`

↓

5

Veredicto

`{ datos }` con `estado` y `vigente`. Si el código no existe: 404.

`JSON`

Diferencia con el listado: `verificarPorCodigo()` es para el público y devuelve la información necesaria para validar el documento; `obtenerCompleto()` es para el panel interno y reúne todos los

exámenes del certificado.

7 Ciclo de vida del certificado

El estado no es un cálculo de la interfaz: vive en la columna `estado` (`ENUM('Vigente','Vencido')`) y se actualiza en la base de datos.



Dos formas de llegar a "Vencido": `actualizarEstado(id, 'Vencido')` para un caso puntual, o `marcarVencidos(fecha)` para actualizar en bloque todos los certificados cuya vigencia ya pasó. La ruta expone `PATCH /api/certificados/marcar-vencidos`.

Regla de integridad: el esquema exige `vigencia >= emision`. Un certificado no puede vencer antes de haber sido emitido.

8 API y reglas que sostienen el sistema

Siete recursos cuelgan de `/api`. Todos los listados aceptan `limite`, `desplazamiento`, `orden` y filtros propios, y responden `{ total, datos }`.

RECURSO	RUTA BASE	MODELO	QUÉ RESUELVE
Estado	`/api/salud`	`conexion.probarConexion`	Comprueba que MySQL responde.
Centros	`/api/centros`	`centros`	Catálogo, subregiones y dependencias.
Personas	`/api/personas`	`personas`	Usuarios, cédula y búsqueda.
Vacunas	`/api/vacunas`	`vacunas`	Catálogo, lotes y obligatoriedad.
Dosis	`/api/dosis`	`dosis`	Aplicaciones, historial y estadísticas.
Certificados	`/api/certificados`	`certificadosSalud`	Emisión, verificación y estado.
Exámenes	`/api/examenes`	`examenes`	Detalle de cada certificado.

Garantías del código

🛡️ SQL parametrizado
Los valores viajan como `?`. Los `ORDER BY` y `LIMIT` se resuelven con listas blancas y números validados.

🔁 camelCase ↔ snake_case
La API usa `primerNombre`, `centroId`; MySQL usa `primer_nombre`, `centro_id`. El mapeo es automático.

🧩 JSON automático
`lotes` y `documentacion` se aceptan como arreglos/objetos JS y se serializan al guardar.

🗓️ Fechas estables
`dateStrings: true` devuelve `YYYY-MM-DD`, sin desfases de zona horaria.

⚠️ Duplicados claros
Un `ER_DUP_ENTRY` se traduce a un error con `codigo: 'DUPLICADO'` para que la ruta responda bien.

🔒 Transacciones
Las operaciones de varias tablas usan `transaccion()`: `commit` si todo va bien, `rollback` si algo falla.

📄 Paginación acotada
`limite` se limita a 200 y nunca se concatena directamente en el SQL.

🧱 Separación de capas
Las rutas no escriben SQL y los modelos no contienen lógica HTTP ni vistas.

🧪 Validación doble

El modelo exige campos y coherencia; MySQL refuerza con FK y CHECK. La interfaz valida antes de enviar.

En una frase: las vistas capturan y muestran datos; las rutas traducen HTTP a llamadas de modelo; los modelos construyen SQL seguro y devuelven promesas; MySQL garantiza la integridad. Cada capa hace una sola cosa.